

MÓDULO 8: SEGURIDAD

SOFTWARE AND DATA INTEGRITY FAILURES

OWASP TOP 10

BIG school



Software and Data Integrity Failures

BIG school

Índice

- **Introducción: cuando el enemigo entra por tu propia puerta**
- **¿Qué significa Software and Data Integrity Failures?**
- **Ejemplos de fallos típicos**
- **¿Qué puede hacer un atacante si explota esto?**
- **¿Cómo detectar este tipo de vulnerabilidades?**
- **¿Cómo prevenir Software and Data Integrity Failures?**
- **Conclusiones**



Software and Data Integrity Failures

BIG school

Introducción: cuando el enemigo entra por tu propia puerta

- Puedes tener código perfecto, pero si una librería o proceso que usas está comprometido, todo falla.
- Esta vulnerabilidad aparece cuando confías en software, datos o integraciones sin verificar su integridad.
- Se aprovecha del exceso de confianza en terceros: dependencias, pipelines o actualizaciones externas.

¿Qué significa Software and Data Integrity Failures?

Software integrity: verificar que lo que instalas o ejecutas no ha sido modificado.

Data integrity: asegurar que los datos no se alteran en tránsito o almacenamiento.

Falla cuando no se valida la autenticidad o consistencia de componentes y datos.

Resultados posibles:

- Dependencias manipuladas (supply chain attack).
- Código malicioso en actualizaciones.
- Archivos de configuración o datos alterados.

Ejemplos de fallos típicos

- Dependencias sin verificar: instalación desde repositorios públicos (npm, Maven, PyPI) sin validar origen.
- Actualizaciones sin firma: descargas automáticas sin verificar autenticidad (suplantación de servidor).
- Pipelines CI/CD confiados: scripts o imágenes descargadas sin hash o firma, vulnerando builds.
- Configuraciones externas manipulables: archivos JSON/YAML alterados para cambiar permisos o claves.
- Backups o datos sin control de integridad: restauraciones con información modificada o inyectada.

¿Qué puede hacer un atacante si explota esto?

- Ejecución remota de código a través de dependencias o pipelines comprometidos.
- Ataques a la cadena de suministro: infectar software legítimo y distribuir malware a usuarios (SolarWinds, CCleaner).
- Manipulación de configuraciones o datos: alterar parámetros críticos para filtrar información.
- Persistencia oculta: mantener código malicioso camuflado en procesos legítimos.

¿Cómo detectar este tipo de vulnerabilidades?

- Revisar pipelines: identificar scripts o descargas de Internet sin validación de origen.
- Verificación de dependencias: usar lockfiles, comparar hashes y monitorizar cambios sospechosos.
- Escaneo en CI/CD: usar herramientas como Trivy, Snyk o Anchore en cada build.
- Monitoreo de integridad: Tripwire o AIDE para detectar modificaciones en archivos críticos.
- Revisión de configuración: validar firmas y autenticidad de archivos antes del despliegue.
- Infraestructura como código: asegurar que los scripts provienen de repos confiables y firmados.

¿Cómo prevenir Software and Data Integrity Failures?

- Firmas digitales
- Control de dependencias
- Seguridad en el pipeline
- Integridad de datos
- Control de acceso
- Contenedores seguros

Conclusiones

1. No basta con escribir buen código, sino con asegurar toda la cadena de confianza.
2. Cada dependencia, script o integración amplía tu superficie de riesgo.
3. Las buenas prácticas de integridad protegen tu software de manipulaciones invisibles.
4. La pregunta clave antes de instalar o desplegar: "¿Estoy verificando la fuente o simplemente confiando?"

Tu responsabilidad comienza cuando decides qué confías, no cuando terminas de programar.